01. Web Foundations

Detailed beginner handbook - English and Arabic

Technologies: HTML, CSS, JavaScript

What you will learn

Build a safe analytics card that changes after a button click.

HTML gives a page structure, CSS gives it appearance, and JavaScript gives it behaviour.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولا، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/01-web-foundations.
3. Read README.md.
4. Run Open index.html in a browser..
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat HTML, CSS, JavaScript as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - How browsers request, parse, and display a page

What is it?

A browser requests files over HTTP, parses HTML into the DOM, parses CSS into style rules, runs JavaScript, then combines layout and paint instructions into pixels.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<!-- Browser starts with this HTML file -->
<h1>Hello, web!</h1>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates How browsers request, parse, and display a page.

هذا الدرس يشرح: How browsers request, parse, and display a page. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Semantic HTML: headings, sections, forms, tables, and accessibility

What is it?

Semantic elements describe purpose: main contains primary content, nav contains navigation, button performs an action, and label names a form control. Meaning improves accessibility and maintenance.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<main>
  <h1>Dashboard</h1>
  <button type="button">Refresh</button>
</main>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Semantic HTML: headings, sections, forms, tables, and accessibility.

هذا الدرس يشرح: Semantic HTML: headings, sections, forms, tables, and accessibility. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - CSS selectors, cascade, inheritance, and specificity

What is it?

Selectors choose elements. The cascade resolves competing rules using origin, importance, specificity, and source order. Inheritance passes selected properties such as font settings to descendants.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
p { color: navy; }
.card p { font-weight: bold; }
#total { font-size: 2rem; }
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates CSS selectors, cascade, inheritance, and specificity.

هذا الدرس يشرح: CSS selectors, cascade, inheritance, and specificity. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - The box model, spacing, colours, typography, Flexbox, and Grid

What is it?

Every element has content, padding, border, and margin. Flexbox arranges items in one dimension; Grid handles rows and columns. Responsive rules adapt layouts to screen width.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
.card { padding: 16px; border: 1px solid #ccc; }
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates The box model, spacing, colours, typography, Flexbox, and Grid.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. The box model, spacing, colours, typography, Flexbox, and Grid :هذا الدرس يشرح

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - JavaScript values, variables, arrays, objects, and functions

What is it?

Values include strings, numbers, booleans, null, arrays, and objects. let stores a changing binding; const prevents reassignment. Functions receive parameters and return reusable results.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const project = 'Learning';
let total = 12;
const rows = [4, 8];
function add(a, b) { return a + b; }
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates JavaScript values, variables, arrays, objects, and functions.

هذا الدرس يشرح: JavaScript values, variables, arrays, objects, and functions. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - The DOM: finding, creating, changing, and removing elements

What is it?

The DOM is the browser's object representation of HTML. querySelector finds an element; textContent changes text; createElement builds safe nodes; append attaches them to the page.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const total = document.querySelector('#total');
total.textContent = '12 records';
const note = document.createElement('p');
note.textContent = 'Synthetic data';
document.body.append(note);
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates The DOM: finding, creating, changing, and removing elements.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. The DOM: finding, creating, changing, and removing elements هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Events: click, input, submit, keyboard, and event propagation

What is it?

Events report user and browser actions. addEventListener connects an event to a handler. preventDefault stops default form navigation; propagation explains how events travel through ancestors.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const button = document.querySelector('#refresh');
button.addEventListener('click', () => {
  console.log('Refresh clicked');
});
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Events: click, input, submit, keyboard, and event propagation.

هذا الدرس يشرح: Events: click, input, submit, keyboard, and event propagation. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Fetch, JSON, promises, async/await, loading, and error states

What is it?

fetch returns a Promise for an HTTP response. Check response.ok, parse JSON, await asynchronous work inside try/catch, and display separate loading, success, empty, and error states.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
async function loadSummary() {
  const response = await fetch('/api/summary');
  if (!response.ok) throw new Error('Request failed');
  return response.json();
}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Fetch, JSON, promises, async/await, loading, and error states.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Fetch, JSON, promises, async/await, loading, and error states هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Responsive design and browser developer tools

What is it?

Use relative units, flexible containers, media queries, and mobile-first rules. Browser developer tools inspect elements, computed styles, network requests, console errors, and responsive layouts.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
@media (max-width: 600px) {
  .grid { grid-template-columns: 1fr; }
}
/* Inspect this rule in browser DevTools. */
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Responsive design and browser developer tools.

هذا الدرس يشرح: Responsive design and browser developer tools. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Accessible, secure, and maintainable frontend habits

What is it?

Keyboard access, visible focus, labels, sufficient contrast, escaped text, validated URLs, small modules, and clear naming make interfaces safer and easier to use.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Web Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<label for="month">Month</label>
<select id="month"><option>January</option></select>
<button aria-label="Download report">Download</button>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Accessible, secure, and maintainable frontend habits.

هذا الدرس يشرح: Accessible, secure, and maintainable frontend habits. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Build a safe analytics card that changes after a button click.

Starting example

```
<button id="add">Add</button>
<p id="total">12 records</p>
<script>let n=12; add.onclick=()=>total.textContent=`${++n} records`</script>
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines HTML, CSS, JavaScript with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة آمنة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step	Next: 02. TypeScript